Comparação computacional dos métodos diretos para sistemas lineares

Número de operações, tempo de execução, memória e precisão numérica

Cálculo Numérico – Licenciatura em Matemática – UFTM

Sumário

1 Introdução **1**

2 Contagem exata de operações **2**

3 Comparação numérica do número de operações **2**

4 Memória **4**

5 Experimento computacional **4**

5.1 Como foi feito 4

5.2 Tempo de execução 5

5.3 Regra de Cramer com cofatores 6

5.4 Matrizes tridiagonais: Thomas contra a LU cheia 6

5.5 Precisão numérica 7

6 Quadro-resumo **9**

1 Introdução

Um **método direto** resolve $Ax = b$ em um número finito de passos e, em aritmética exata, fornece a solução exata. A diferença entre os métodos está no *custo* (quantas operações e quanto tempo), na *memória* e no comportamento diante dos *erros de arredondamento*. Este texto compara os seguintes métodos:

- eliminação gaussiana, sem e com pivoteamento parcial;
- método de Gauss-Jordan;
- fatoração LU (com $PA = LU$);
- fatoração de Cholesky ($A = LL^T$) e a variante LDL^T;
- fatoração QR, por Householder, Gram-Schmidt modificado e Givens;
- cálculo da matriz inversa ($x = A^{-1}b$);
- regra de Cramer, com determinantes por eliminação e por cofatores;

- algoritmo de Thomas, para matrizes tridiagonais.

A comparação é feita de duas formas. Na primeira (Seções 2 a 4), contamos as operações de ponto flutuante com fórmulas exatas. Na segunda (Seção 5), todos os métodos foram programados, e o programa contou as operações que de fato executou, mediu o tempo e o erro numérico.

Convenção. Contamos separadamente multiplicações e divisões (M/D), adições e subtrações (A/S) e raízes quadradas. O *total* é a soma das três contagens. A matriz é $n \times n$ e, salvo menção em contrário, há um único vetor b.

2 Contagem exata de operações

A Tabela 1 reúne as fórmulas. As do grupo superior são exatas e foram conferidas pelo programa da Seção 5: para $n = 4$, 10 e 20, a contagem feita pelo programa coincidiu, número por número, com cada fórmula. Para a QR, indicamos o termo principal.

Método	Multiplicações/divisões	Adições/subtrações	Raízes
Eliminação gaussiana (com retrossubstituição)	$\frac{n^3}{3} + n^2 - \frac{n}{3}$	$\frac{n^3}{3} + \frac{n^2}{2} - \frac{5n}{6}$	—
Gauss-Jordan	$\frac{n^3}{2} + n^2 - \frac{n}{2}$	$\frac{n^3}{2} - \frac{n}{2}$	—
LU: fatoração	$\frac{n^3-n}{3}$	$\frac{n^3}{3} - \frac{n^2}{2} + \frac{n}{6}$	—
LU: resolução ($Ly = Pb$, $Ux = y$)	n^2	$n^2 - n$	—
Cholesky: fatoração	$\frac{n^3}{6} + \frac{n^2}{2} - \frac{2n}{3}$	$\frac{n^3-n}{6}$	n
Cholesky: resolução ($Ly = b$, $L^T x = y$)	$n^2 + n$	$n^2 - n$	—
LDL^T: fatoração	$\frac{n^3}{6} + n^2 - \frac{7n}{6}$	$\frac{n^3-n}{6}$	—
LDL^T: resolução	n^2	$n^2 - n$	—
Matriz inversa (Gauss-Jordan) e produto $A^{-1}b$	$n^3 + n^2$	$n^3 - n^2$	—
Cramer, determinantes por eliminação	$(n+1)\left(\frac{n^3-n}{3} + n - 1\right) + n$	$(n+1)\frac{(n-1)n(2n-1)}{6}$	—
Algoritmo de Thomas (tridiagonal)	$5n - 4$	$3n - 3$	—
QR por Householder	$\approx \frac{2}{3}n^3$	$\approx \frac{2}{3}n^3$	$n - 1$
QR por Gram-Schmidt modificado	$\approx n^3$	$\approx n^3$	n
QR por Givens	$\approx \frac{4}{3}n^3$	$\approx \frac{2}{3}n^3$	$\frac{n(n-1)}{2}$
Cramer, determinantes por cofatores	$M_n = nM_{n-1} + n$,	$S_n = nS_{n-1} + n - 1$;	total $\approx e\,(n+1)!$

Tabela 1: Número de operações para resolver $Ax = b$ com A de ordem n.

Somando as colunas e guardando o termo principal, obtemos a **ordem de custo** de cada método:

$$\underbrace{8n}_{\text{Thomas}} \ll \underbrace{\tfrac{1}{3}n^3}_{\text{Cholesky, } LDL^T} < \underbrace{\tfrac{2}{3}n^3}_{\text{Gauss, LU}} < \underbrace{n^3}_{\text{Gauss-Jordan}} < \underbrace{\tfrac{4}{3}n^3}_{\text{QR Householder}} < \underbrace{2n^3}_{\substack{\text{inversa, QR por} \\ \text{Gram-Schmidt e Givens}}} \ll \underbrace{\tfrac{2}{3}n^4}_{\text{Cramer (elim.)}} \ll \underbrace{e\,(n+1)!}_{\text{Cramer (cof.)}}$$

3 Comparação numérica do número de operações

A Tabela 2 e a Figura 1 mostram os totais para alguns valores de n. Os valores da QR foram contados pelo programa; os demais vêm das fórmulas.

Método	$n = 4$	$n = 10$	$n = 20$	$n = 100$	$n = 1000$
Thomas (tridiagonal)	25	73	153	793	7 993
Cholesky	62	585	3 670	358 350	$3{,}4 \times 10^{8}$
LDL^T	60	610	3 820	363 100	$3{,}4 \times 10^{8}$
Eliminação gaussiana / LU	62	805	5 910	681 550	$6{,}7 \times 10^{8}$
Gauss-Jordan	76	1 090	8 380	1 009 900	$1{,}0 \times 10^{9}$
QR (Householder)	207	2 077	13 567	1 403 887	$1{,}3 \times 10^{9}$
QR (Gram-Schmidt mod.)	214	2 545	18 190	2 054 950	$2{,}0 \times 10^{9}$
QR (Givens)	208	2 620	18 640	2 069 200	$2{,}0 \times 10^{9}$
Matriz inversa	128	2 000	16 000	2 000 000	$2{,}0 \times 10^{9}$
Cramer (det. por eliminação)	189	6 874	108 149	$6{,}7 \times 10^{7}$	$6{,}7 \times 10^{11}$
Cramer (det. por cofatores)	319	$1{,}1 \times 10^{8}$	$1{,}4 \times 10^{20}$	$2{,}6 \times 10^{160}$	$1{,}1 \times 10^{2571}$

Tabela 2: Total de operações (M/D + A/S + raízes). Para o algoritmo de Thomas a matriz é tridiagonal; nos demais, cheia.

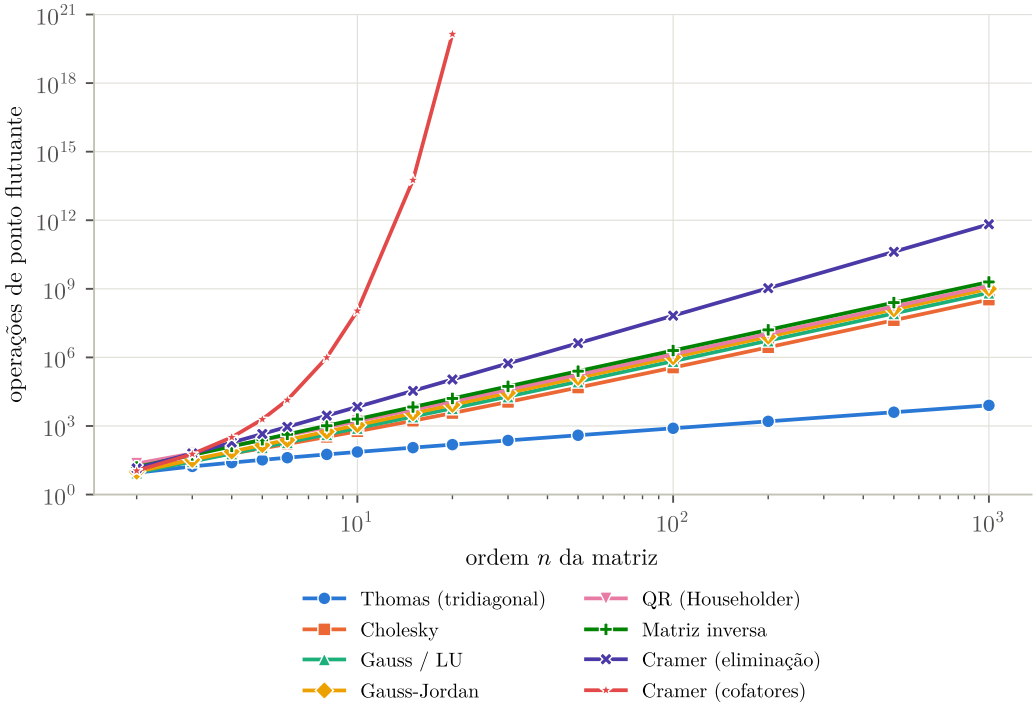


Figura 1: Total de operações em função de n (escalas logarítmicas). Retas paralelas indicam a mesma potência de n: inclinação 1 para Thomas, 3 para os métodos de ordem n^3 e 4 para Cramer por eliminação. A curva de Cramer por cofatores não é reta: seu crescimento é fatorial.

Num computador de 10^{12} operações por segundo, com $n = 1000$, a eliminação gaussiana ($6{,}7 \times 10^{8}$ operações) leva menos de um milésimo de segundo; Cramer por eliminação ($6{,}7 \times 10^{11}$) leva quase um segundo; Cramer por cofatores levaria um tempo sem qualquer sentido físico.

Vários vetores b com a mesma matriz

Quando a mesma matriz A aparece com m vetores $b_1, \ldots, b_m$ que só ficam conhecidos aos poucos (por exemplo, em um processo iterativo), as fatorações levam grande vantagem: a parte cara é feita

uma única vez.

Método	Custo para m sistemas	Comentário
Eliminação gaussiana repetida	$m \cdot \frac{2}{3}n^3$	refaz a eliminação a cada b
LU	$\frac{2}{3}n^3 + 2mn^2$	fatora uma vez
Cholesky ou LDL^T	$\frac{1}{3}n^3 + 2mn^2$	só para matriz simétrica
QR (Householder)	$\frac{4}{3}n^3 + 3mn^2$	aplica Q^T e retrossubstitui
Matriz inversa	$2n^3 + 2mn^2$	mais cara que a LU para qualquer m
Cramer (eliminação)	$\frac{2}{3}n^4 \cdot m$ (aprox.)	cada b exige n novos determinantes
Thomas (tridiagonal)	$3n + 5mn$	fatoração e resolução lineares em n

Tabela 3: Custo aproximado para m vetores b com a mesma matriz A.

Se todos os vetores b forem conhecidos de uma vez, a eliminação gaussiana aplicada à matriz $[A \,|\, b_1 \cdots b_m]$ tem o mesmo custo da LU. Mesmo com muitos vetores, calcular A^{-1} nunca compensa: a LU resolve cada novo sistema com os mesmos $2n^2$ operações de $A^{-1}b$, e a fatoração custa um terço da inversa.

4 Memória

Método	Memória (números reais)	Observação
Gauss, Gauss-Jordan, LU	$n^2 + n$	L e U cabem no lugar de A
Cholesky, LDL^T	$\frac{n(n+1)}{2} + n$	só a metade inferior
QR (Householder, Givens)	$n^2 + 2n$	Q guardada implicitamente pelos vetores de reflexão ou rotação
QR (Gram-Schmidt)	$2n^2$	Q e R guardadas separadamente
Matriz inversa	$2n^2$	trabalha com $[A \,|\, I]$
Cramer	$2n^2$	cada A_i é uma cópia modificada de A
Thomas	$4n$	só as três diagonais e b

Tabela 4: Memória necessária.

5 Experimento computacional

5.1 Como foi feito

Todos os métodos foram programados em Python 3.13 com a biblioteca NumPy 2.5, num processador Intel Xeon de 2,1 GHz com 2 núcleos. Em cada passo, as operações com linhas e colunas são feitas de uma vez pelo NumPy (por exemplo, a atualização de toda a submatriz na eliminação), e um contador acumula exatamente o número de operações escalares de cada bloco. Para comparação, também foi medida a função `numpy.linalg.solve`, que usa a biblioteca LAPACK (fatoração LU com pivoteamento parcial, escrita em Fortran e otimizada).

Os testes usam matrizes aleatórias com solução conhecida $x^* = (1, \ldots, 1)$, isto é, $b = Ax^*$. Nos testes de tempo a matriz é simétrica definida positiva ($A = BB^T + nI$, com B aleatória), para que todos os métodos, inclusive Cholesky e LDL^T, possam ser comparados na mesma matriz. Cada tempo é o menor de três execuções (uma execução para $n \geq 400$).

5.2 Tempo de execução

Método	$n = 50$	$n = 100$	$n = 200$	$n = 400$	$n = 800$	Fator
Gauss sem pivoteamento	751 µs	2,1 ms	6,9 ms	75,1 ms	546,5 ms	7,3
Gauss com pivoteamento parcial	990 µs	2,7 ms	10,5 ms	72,5 ms	532,4 ms	7,3
LU, atualizações de posto 1	712 µs	2,0 ms	8,1 ms	66,3 ms	532,7 ms	8,0
LU, por linhas e colunas (Crout)	586 µs	1,7 ms	4,0 ms	11,7 ms	66,8 ms	5,7
Gauss-Jordan	792 µs	2,8 ms	12,6 ms	88,0 ms	751,1 ms	8,5
Cholesky	413 µs	876 µs	2,0 ms	4,9 ms	22,4 ms	4,6
LDL^T	426 µs	844 µs	1,9 ms	5,2 ms	22,2 ms	4,3
QR (Householder)	1,1 ms	2,9 ms	10,6 ms	63,9 ms	536,2 ms	8,4
QR (Gram-Schmidt mod. em $[A \mid b]$)	856 µs	2,6 ms	13,0 ms	92,2 ms	778,9 ms	8,4
QR (Gram-Schmidt mod., $Q^T b$ à parte)	3,5 ms	14,7 ms	94,0 ms	428,0 ms	2,24 s	5,2
QR (Givens)	7,8 ms	30,7 ms	228,1 ms	980,5 ms	3,38 s	3,4
Matriz inversa	1,3 ms	5,5 ms	33,8 ms	406,3 ms	2,67 s	6,6
Cramer (det. por eliminação)	26,7 ms	170,5 ms	1,52 s	—	—	8,9*
`numpy.linalg.solve` (LAPACK)	20 µs	79 µs	294 µs	1,6 ms	10,0 ms	6,2

Tabela 5: Tempo de execução. A coluna "Fator" é a razão entre os tempos com $n = 800$ e $n = 400$ (para Cramer, marcado com *, entre $n = 200$ e $n = 100$). Para um método de ordem n^3, o fator esperado é $2^3 = 8$; para ordem n^4, $2^4 = 16$.

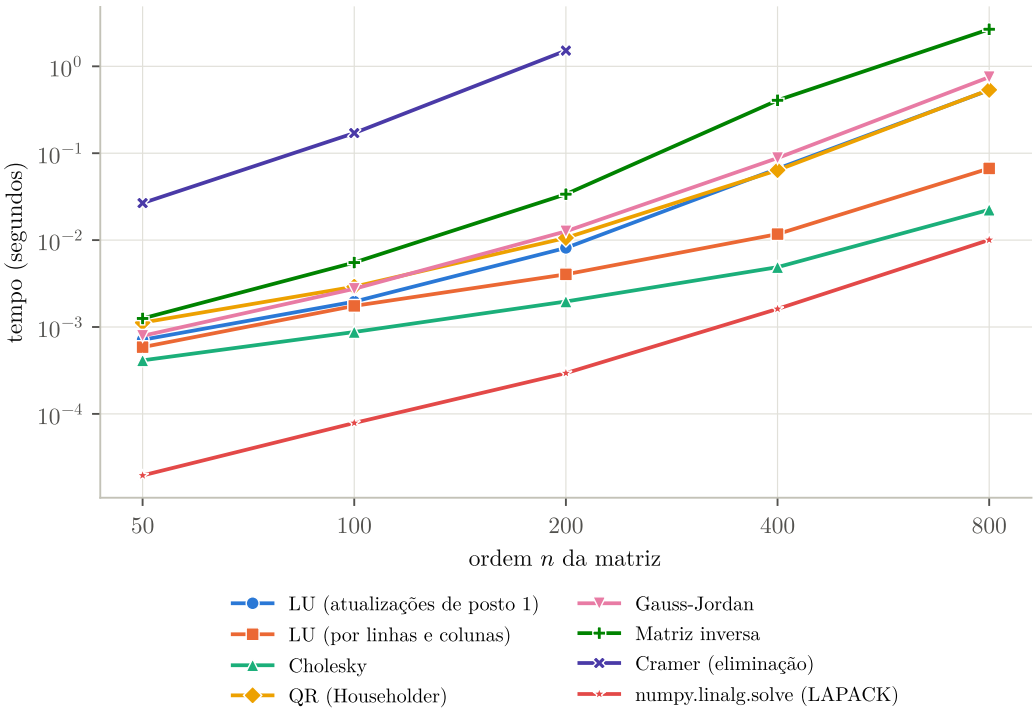


Figura 2: Tempo de execução medido, em escalas logarítmicas.

O que a Tabela 5 e a Figura 2 mostram:

1. **A ordem de crescimento se confirma.** Ao dobrar n de 400 para 800, o tempo dos métodos organizados por atualizações da submatriz (Gauss, LU de posto 1, Gauss-Jordan, QR por Householder e por Gram-Schmidt em $[A \mid b]$) ficou multiplicado por um fator entre 7,3 e 8,5,

perto do fator teórico 8. Fatores menores (Cholesky, LDL^T, LU por Crout, Givens) indicam que, nesses tamanhos, ainda pesa o custo fixo de cada passo do laço em Python.

2. **O número de operações não é tudo.** As duas versões da LU fazem *exatamente* as mesmas operações (a contagem é idêntica), mas a versão organizada por linhas e colunas, com produtos matriz-vetor, foi cerca de **8 vezes** mais rápida que a versão com atualizações de posto 1. A diferença está na forma de acessar a memória: a atualização de posto 1 reescreve a submatriz inteira a cada passo. Pela mesma razão, a Cholesky, organizada como a LU de Crout, ficou cerca de 3 vezes mais rápida que ela, perto do fator 2 previsto pela contagem.

3. **Bibliotecas otimizadas fazem diferença.** A `numpy.linalg.solve` faz a mesma eliminação com pivoteamento parcial ($\frac{2}{3}n^3$ operações), mas foi cerca de 50 vezes mais rápida que a eliminação gaussiana programada aqui. O LAPACK organiza a eliminação em blocos, que aproveitam a memória rápida do processador, e é compilado.

4. **Inversa e Gauss-Jordan custam mais.** A inversa (cerca de $2n^3$) levou cerca de 3,6 vezes o tempo de Gauss-Jordan (n^3), que por sua vez levou cerca de 1,4 vez o da eliminação gaussiana ($\frac{2}{3}n^3$).

5. **Givens e Gram-Schmidt com Q^Tb à parte** executam muitas operações pequenas, uma rotação ou projeção de cada vez, e por isso ficaram lentos em Python, além de fazerem mais operações que Householder.

6. **Cramer por eliminação** já levou 1,5 s com $n = 200$, contra 10,5 ms da eliminação gaussiana com pivoteamento: cerca de 145 vezes mais. Por isso foi medido só até $n = 200$.

5.3 Regra de Cramer com cofatores

n	Operações	Razão com o n anterior	Tempo
3	59	—	285 µs
4	319	5,4	718 µs
5	1 949	6,1	4,2 ms
6	13 691	7,0	28,5 ms
7	109 591	8,0	413,9 ms
8	986 399	9,0	2,43 s

Tabela 6: Regra de Cramer com determinantes por cofatores. A cada aumento de n em uma unidade, o número de operações fica multiplicado por cerca de $n + 1$.

Com $n = 8$, a regra de Cramer por cofatores já fez quase um milhão de operações e levou 2,4 s; a eliminação gaussiana, para o mesmo sistema, faz 428 operações. Multiplicando o tempo por $n+1$ a cada passo, nesta máquina $n = 11$ levaria cerca de uma hora, $n = 12$ quase meio dia e $n = 15$ mais de quatro anos.

5.4 Matrizes tridiagonais: Thomas contra a LU cheia

Para a matriz tridiagonal com diagonal 2,01 e vizinhas -1 (do tipo que aparece na discretização de equações diferenciais), o algoritmo de Thomas foi comparado com a LU aplicada à matriz cheia:

	Thomas		LU na matriz cheia		
n	Operações	Tempo	Operações	Tempo	Razão dos tempos
100	793	121 µs	681 550	3,0 ms	25
400	3 193	481 µs	$4{,}3 \times 10^7$	65,5 ms	136
1 600	12 793	1,9 ms	$2{,}7 \times 10^9$	3,71 s	1983

Tabela 7: Aproveitar a estrutura da matriz muda a ordem do custo de n^3 para n.

Com $n = 1600$, o algoritmo de Thomas foi quase 2000 vezes mais rápido. A LU cheia gasta quase todo o tempo multiplicando e somando zeros.

5.5 Precisão numérica

Medimos o erro relativo $\|x - x^*\|/\|x^*\|$, onde x é a solução calculada. Em aritmética de dupla precisão, o arredondamento unitário é $\varepsilon \approx 1{,}1 \times 10^{-16}$, e um método estável comete erro da ordem de $\varepsilon\,\kappa(A)$, onde $\kappa(A)$ é o número de condição da matriz.

Matrizes bem condicionadas ($n = 200$).

Método	Matriz geral, $\kappa \approx 4{,}1 \times 10^2$	Matriz SPD, $\kappa \approx 7{,}6 \times 10^2$
Gauss sem pivoteamento	$1{,}9 \times 10^{-12}$	$2{,}1 \times 10^{-14}$
Gauss com pivoteamento parcial	$2{,}1 \times 10^{-14}$	$2{,}1 \times 10^{-14}$
Gauss-Jordan	$2{,}1 \times 10^{-14}$	$2{,}1 \times 10^{-14}$
LU, atualizações de posto 1	$2{,}0 \times 10^{-14}$	$2{,}0 \times 10^{-14}$
LU, por linhas e colunas (Crout)	$7{,}0 \times 10^{-15}$	$1{,}5 \times 10^{-14}$
Cholesky	não se aplica	$1{,}3 \times 10^{-14}$
LDL^T	não se aplica	$1{,}3 \times 10^{-14}$
QR (Householder)	$1{,}4 \times 10^{-14}$	$3{,}7 \times 10^{-14}$
QR (Gram-Schmidt mod. em $[A \mid b]$)	$6{,}2 \times 10^{-15}$	$3{,}3 \times 10^{-14}$
QR (Gram-Schmidt mod., Q^Tb à parte)	$1{,}7 \times 10^{-13}$	$4{,}7 \times 10^{-12}$
QR (Givens)	$1{,}6 \times 10^{-14}$	$8{,}0 \times 10^{-14}$
Matriz inversa	$2{,}7 \times 10^{-14}$	$2{,}6 \times 10^{-14}$
Cramer (det. por eliminação)	$2{,}9 \times 10^{-14}$	*estouro (NaN)*
`numpy.linalg.solve` (LAPACK)	$2{,}1 \times 10^{-14}$	$1{,}3 \times 10^{-14}$

Tabela 8: Erro relativo com $n = 200$.

Com matrizes bem condicionadas, todos os métodos estáveis acertam cerca de 13 a 14 algarismos. Três observações:

- a eliminação *sem* pivoteamento perdeu cerca de dois algarismos na matriz geral (erro 100 vezes maior), mas não na matriz SPD, para a qual o pivoteamento não é necessário;
- o Gram-Schmidt modificado com Q^Tb calculado à parte perdeu de um a dois algarismos e meio;
- a regra de Cramer **falhou** na matriz SPD: o determinante de uma matriz 200×200 com entradas da ordem de 200 passa de 10^{308}, o maior número representável, e a divisão $\det(A_i)/\det(A)$ vira ∞/∞. Os outros métodos nunca calculam o determinante e não têm esse problema.

Matrizes mal condicionadas: a matriz de Hilbert. A matriz de Hilbert, $h_{ij} = 1/(i + j - 1)$,

é simétrica definida positiva e muito mal condicionada: $\kappa(H_n)$ cresce cerca de 30 vezes a cada aumento de n.

Método	$n = 4$	$n = 6$	$n = 8$	$n = 10$	$n = 12$	$n = 13$
$\kappa(H_n)$	$1{,}6 \times 10^{4}$	$1{,}5 \times 10^{7}$	$1{,}5 \times 10^{10}$	$1{,}6 \times 10^{13}$	$1{,}6 \times 10^{16}$	$4{,}7 \times 10^{17}$
Gauss sem pivoteamento	$7{,}6 \times 10^{-14}$	$1{,}4 \times 10^{-10}$	$1{,}5 \times 10^{-7}$	$4{,}6 \times 10^{-5}$	$2{,}0 \times 10^{-1}$	$3{,}1 \times 10^{0}$
Gauss com pivoteamento parcial	$9{,}6 \times 10^{-14}$	$1{,}9 \times 10^{-10}$	$3{,}6 \times 10^{-8}$	$8{,}2 \times 10^{-5}$	$2{,}4 \times 10^{-1}$	$2{,}2 \times 10^{0}$
Cholesky	$3{,}0 \times 10^{-13}$	$1{,}0 \times 10^{-10}$	$1{,}3 \times 10^{-7}$	$1{,}0 \times 10^{-4}$	$1{,}8 \times 10^{-1}$	$4{,}1 \times 10^{1}$
LDL^T	$1{,}5 \times 10^{-13}$	$3{,}8 \times 10^{-10}$	$3{,}3 \times 10^{-8}$	$9{,}9 \times 10^{-5}$	$1{,}5 \times 10^{-1}$	$4{,}9 \times 10^{0}$
QR (Householder)	$1{,}6 \times 10^{-13}$	$2{,}9 \times 10^{-11}$	$2{,}4 \times 10^{-7}$	$2{,}5 \times 10^{-5}$	$2{,}0 \times 10^{-1}$	$2{,}2 \times 10^{0}$
QR (Gram-Schmidt mod. em $[A \mid b]$)	$3{,}7 \times 10^{-13}$	$3{,}3 \times 10^{-11}$	$1{,}2 \times 10^{-7}$	$3{,}2 \times 10^{-5}$	$5{,}0 \times 10^{-1}$	$3{,}4 \times 10^{1}$
QR (Gram-Schmidt mod., $Q^T b$ à parte)	$3{,}8 \times 10^{-9}$	$8{,}6 \times 10^{-3}$	$1{,}8 \times 10^{3}$	$2{,}3 \times 10^{9}$	$1{,}4 \times 10^{15}$	$8{,}2 \times 10^{16}$
QR (Givens)	$4{,}5 \times 10^{-13}$	$8{,}6 \times 10^{-11}$	$1{,}2 \times 10^{-7}$	$1{,}9 \times 10^{-4}$	$3{,}3 \times 10^{-1}$	$3{,}9 \times 10^{0}$
Matriz inversa	$4{,}5 \times 10^{-13}$	$2{,}7 \times 10^{-10}$	$3{,}9 \times 10^{-7}$	$3{,}0 \times 10^{-4}$	$2{,}9 \times 10^{-1}$	$2{,}8 \times 10^{0}$
Cramer (det. por eliminação)	$9{,}9 \times 10^{-14}$	$1{,}9 \times 10^{-10}$	$3{,}8 \times 10^{-8}$	$8{,}2 \times 10^{-5}$	$2{,}4 \times 10^{-1}$	$2{,}3 \times 10^{0}$

Tabela 9: Erro relativo na resolução de $H_n x = H_n \mathbf{1}$.

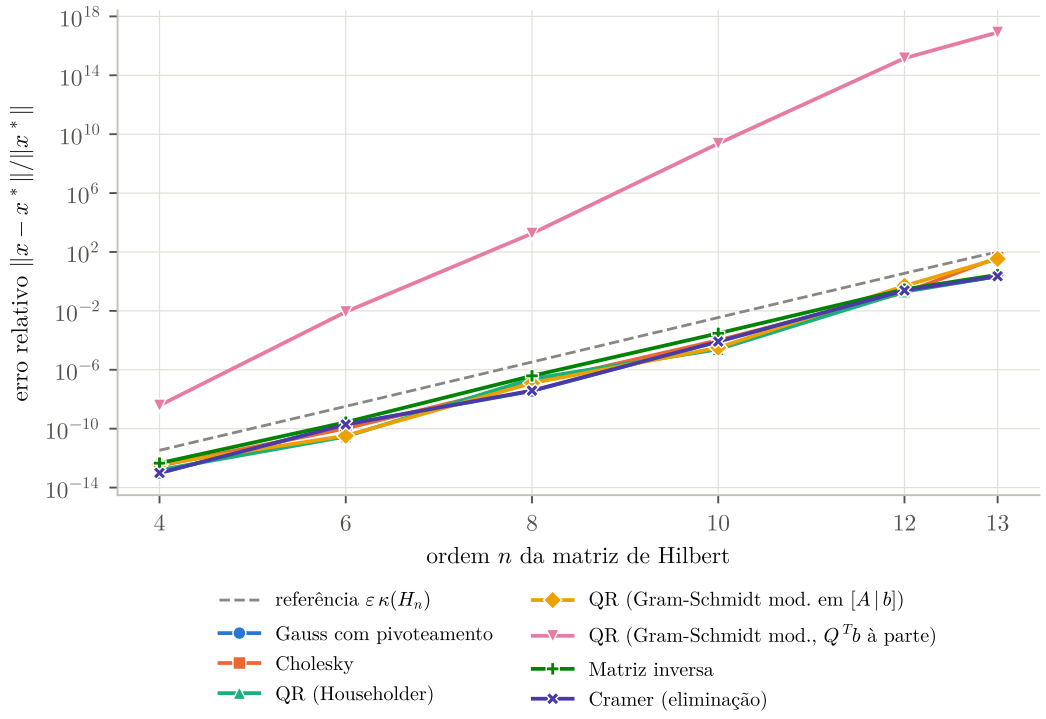


Figura 3: Erro relativo na matriz de Hilbert. A linha tracejada é a estimativa $\varepsilon\,\kappa(H_n)$.

A Tabela 9 e a Figura 3 mostram que:

- todos os métodos estáveis seguem a reta $\varepsilon\,\kappa(H_n)$: com $n = 12$ o erro já é de 20% e com $n = 13$ a solução calculada não tem nenhum algarismo correto. O problema é o *condicionamento da matriz*, e nenhum método direto o resolve;
- o Gram-Schmidt modificado com $Q^T b$ calculado à parte segue $\varepsilon\,\kappa^2$: as colunas de Q perdem ortogonalidade, e o erro já é de 10^3 com $n = 8$. Aplicado à matriz aumentada $[A \mid b]$, o mesmo algoritmo volta a ser estável.

Por que pivotear. Para $A = \begin{pmatrix} 10^{-17} & 1 \\ 1 & 1 \end{pmatrix}$ e $b = (1, 2)$, a solução é $x \approx (1, 1)$. Sem pivoteamento, o multiplicador é $m_{21} = 10^{17}$, a segunda equação vira $-10^{17}x_2 = -10^{17}$ (os números 1 e 2 desaparecem no arredondamento) e o programa obteve $x = (0, 1)$: o valor de x_1 está *totalmente* errado. Com pivoteamento parcial as linhas são trocadas e o programa obteve $x = (1, 1)$.

6 Quadro-resumo

Método	Custo	Quando se aplica	Estabilidade e observações
Thomas	$8n$	tridiagonal (ou banda)	estável para matriz diagonalmente dominante ou SPD
Cholesky	$\frac{1}{3}n^3$	simétrica definida positiva	estável sem pivoteamento; serve de teste de SPD
LDL^T	$\frac{1}{3}n^3$	simétrica	sem raízes; para matriz indefinida precisa de pivoteamento especial
Gauss sem pivô	$\frac{2}{3}n^3$	SPD ou diagonalmente dominante	instável em geral (pivô pequeno)
Gauss / LU com pivoteamento	$\frac{2}{3}n^3$	qualquer matriz invertível	estável na prática; LU reaproveita a fatoração para vários b
Gauss-Jordan	n^3	qualquer	50% mais caro que Gauss, sem ganho de precisão
QR (Householder)	$\frac{4}{3}n^3$	qualquer, inclusive $m \times n$	sempre estável; padrão para mínimos quadrados
QR (Givens)	$2n^3$	matriz esparsa ou de Hessenberg	estável; zera um elemento por vez
QR (Gram-Schmidt mod.)	$2n^3$	qualquer	estável só aplicado a $[A \mid b]$
Matriz inversa	$2n^3$	qualquer	3 vezes mais cara que a LU; não use para resolver $Ax = b$
Cramer (eliminação)	$\frac{2}{3}n^4$	qualquer	n vezes mais cara; o determinante pode estourar
Cramer (cofatores)	$e\,(n+1)!$	só $n \leq 3$	inviável já para $n \approx 12$

Tabela 10: Resumo da comparação.

Como escolher o método.

1. Matriz tridiagonal ou de banda: algoritmo de Thomas ou LU de banda.
2. Matriz simétrica definida positiva: Cholesky.
3. Matriz quadrada geral: LU com pivoteamento parcial; com vários vetores b, reaproveite a fatoração.
4. Sistemas sobredeterminados (mínimos quadrados) ou matriz próxima de singular: QR por Householder.
5. Matriz inversa e regra de Cramer: não use para calcular a solução. Elas têm valor teórico (fórmulas fechadas, demonstrações, sistemas 2×2 e 3×3).
6. Na prática, use uma biblioteca (LAPACK, NumPy, MATLAB, Octave): o algoritmo é o mesmo, mas a implementação é dezenas de vezes mais rápida.

Programas. As implementações e o experimento estão nos arquivos `metodos.py` e `experimento.py`, que acompanham este texto. Como as matrizes são geradas aleatoriamente e os tempos dependem da máquina, uma nova execução dá números um pouco diferentes, mas as mesmas conclusões.

Referências

[1] R. L. Burden, J. D. Faires, A. M. Burden. *Análise Numérica*. 3. ed. São Paulo: Cengage Learning, 2016.

[2] M. A. G. Ruggiero, V. L. R. Lopes. *Cálculo Numérico: aspectos teóricos e computacionais*. 2. ed. São Paulo: Pearson Makron Books, 1996.

[3] G. H. Golub, C. F. Van Loan. *Matrix Computations*. 4. ed. Baltimore: Johns Hopkins University Press, 2013.

[4] L. N. Trefethen, D. Bau III. *Numerical Linear Algebra*. Philadelphia: SIAM, 1997.

[5] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*. 2. ed. Philadelphia: SIAM, 2002.